

uniq

diff

echo

chown

chmod

umask

du

df

basename

dirname

ps

top

kill

killall

jobs

bg

fg

type

which

nohup

xargs

vim

emacs

nano

whoami

who

su

sudo

passwd

ping

traceroute

clear

history

export

crontab

uname

env

printenv

Conclusion

Preface

The Linux Commands Handbook follows the 80/20 rule: learn in 20% of the time the 80% of a topic.

I find this approach gives a well-rounded overview.

This book does not try to cover everything under the sun related to Linux and its commands. It focuses on the small core commands that you will use the 80% or 90% of the time, trying to simplify the usage of the more complex ones.

All those commands work on Linux, macOS, WSL, and anywhere you have a UNIX environment.

I hope the contents of this book will help you achieve what you want: **get comfortable with Linux**.

This book is written by Flavio. I **publish programming tutorials** every day on my website flaviocopes.com.

You can reach me on Twitter [@flaviocopes](https://twitter.com/flaviocopes).

Enjoy!

Introduction to Linux and shells

Linux is an operating system, like macOS or Windows.

It is also the most popular Open Source and free, as in freedom, operating system.

It powers the vast majority of the servers that compose the Internet. It's the base upon which everything is built upon. But not just that. Android is based on (a modified version of) Linux.

The Linux "core" (called *kernel*) was born in 1991 in Finland, and it went a really long way from its humble beginnings. It went on to be the kernel of the GNU Operating System, creating the duo GNU/Linux.

There's one thing about Linux that corporations like Microsoft and Apple, or Google, will never be able to offer: the freedom to do whatever you want with your computer.

They're actually going in the opposite direction, building walled gardens, especially on the mobile side.

Linux is the ultimate freedom.

It is developed by volunteers, some paid by companies that rely on it, some independently, but there's no single commercial company that can dictate what goes into Linux, or the project priorities.

Linux can also be used as your day to day computer. I use macOS because I really enjoy the applications, the design and I also used to be an iOS and Mac apps developer, but before using it I used Linux as my main computer Operating System.

No one can dictate which apps you can run, or "call home" with apps that track you, your position, and more.

Linux is also special because there's not just "one Linux", like it happens on Windows or macOS. Instead, we have **distributions**.

A "distro" is made by a company or organization and packages the Linux core with additional programs and tooling.

For example you have Debian, Red Hat, and Ubuntu, probably the most popular.

Many, many more exist. You can create your own distribution, too. But most likely you'll use a popular one, one that has lots of users and a community of people around it, so you can do what you need to do without losing too much time reinventing the wheel and figuring out answers to common problems.

Some desktop computers and laptops ship with Linux preinstalled. Or you can install it on your Windows-based computer, or on a Mac.

But you don't need to disrupt your existing computer just to get an idea of how Linux works.

I don't have a Linux computer.

If you use a Mac you need to know that under the hood macOS is a UNIX Operating System, and it shares a lot of the same ideas and software that a GNU/Linux system uses, because GNU/Linux is a free alternative to UNIX.

UNIX is an umbrella term that groups many operating systems used in big corporations and institutions, starting from the 70's

The macOS terminal gives you access to the same exact commands I'll describe in the rest of this handbook.

Microsoft has an official [Windows Subsystem for Linux](#) which you can (and should!) install on Windows. This will give you the ability to run Linux in a very easy way on your PC.

But the vast majority of the time you will run a Linux computer in the cloud via a VPS (Virtual Private Server) like DigitalOcean.

A shell is a command interpreter that exposes to the user an interface to work with the underlying operating system.

It allows you to execute operations using text and commands, and it provides users advanced features like being able to create scripts.

This is important: shells let you perform things in a more optimized way than a GUI (Graphical User Interface) could ever possibly let you do. Command line tools can offer many different configuration options without being too complex to use.

There are many different kind of shells. This post focuses on Unix shells, the ones that you will find commonly on Linux and macOS computers.

Many different kind of shells were created for those systems over time, and a few of them dominate the space: Bash, Csh, Zsh, Fish and many more!

All shells originate from the Bourne Shell, called `sh`. "Bourne" because its creator was Steve Bourne.

Bash means *Bourne-again shell*. `sh` was proprietary and not open source, and Bash was created in 1989 to create a free alternative for the GNU project and the Free Software Foundation. Since projects had to pay to use the Bourne shell, Bash became very popular.

If you use a Mac, try opening your Mac terminal. That by default is running ZSH. (or, pre-Catalina, Bash)

You can set up your system to run any kind of shell, for example I use the Fish shell.

Each single shell has its own unique features and advanced usage, but they all share a common functionality: they can let you execute programs, and they can be programmed.

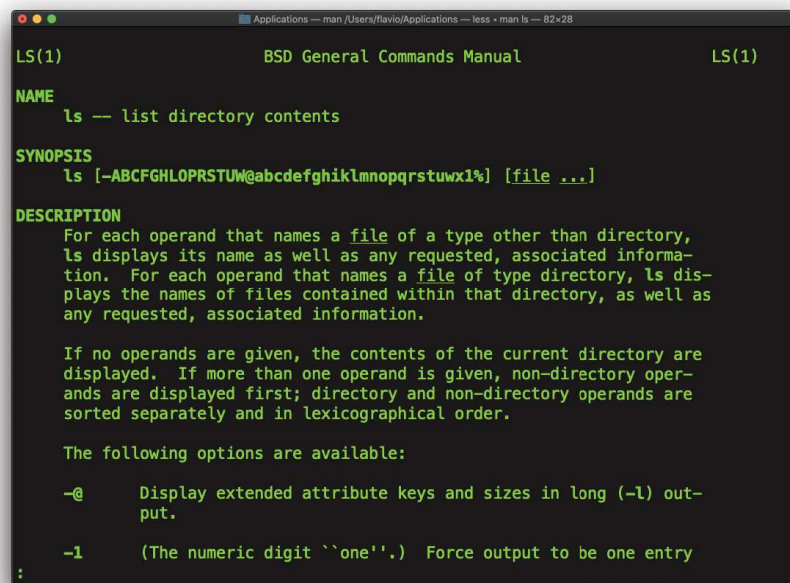
In the rest of this handbook we'll see in detail the most common commands you will use.

man

The first command I want to introduce is a command that will help you understand all the other commands.

Every time I don't know how to use a command, I type

`man <command>` to get the manual:



```
LS(1) BSD General Commands Manual LS(1)

NAME
  ls — list directory contents

SYNOPSIS
  ls [-ABCFGHLOPRSTUW@abcdefghiklmnopqrstuwx1] [file ...]

DESCRIPTION
  For each operand that names a file of a type other than directory,
  ls displays its name as well as any requested, associated informa-
  tion. For each operand that names a file of type directory, ls dis-
  plays the names of files contained within that directory, as well as
  any requested, associated information.

  If no operands are given, the contents of the current directory are
  displayed. If more than one operand is given, non-directory oper-
  ands are displayed first; directory and non-directory operands are
  sorted separately and in lexicographical order.

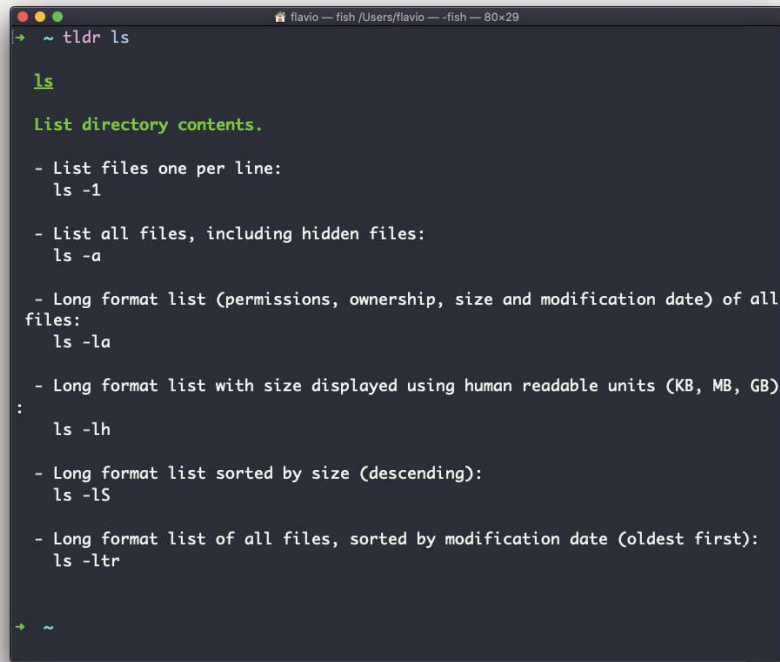
  The following options are available:

  -@      Display extended attribute keys and sizes in long (-l) out-
          put.
  -1      (The numeric digit ``one''.) Force output to be one entry
  :
```

This is a man (from *manual*) page. Man pages are an essential tool to learn, as a developer. They contain so much information that sometimes it's almost too much.

The above screenshot is just 1 of 14 screens of explanation for the `ls` command.

Most of the times when I'm in need to learn a command quickly I use this site called **tldr pages**: <https://tldr.sh/>. It's a command you can install, then you run it like this: `tldr <command>`, which gives you a very quick overview of a command, with some handy examples of common usage scenarios:

A terminal window with a dark background and light text. The title bar at the top shows a home icon, the name 'flavio', and the path '— fish /Users/flavio — -fish — 80x29'. The prompt is '~ tldr ls'. The output shows the command 'ls' in green, followed by 'List directory contents.' in green. Then, a list of options for 'ls' is shown in white text: '- List files one per line: ls -l', '- List all files, including hidden files: ls -a', '- Long format list (permissions, ownership, size and modification date) of all files: ls -la', '- Long format list with size displayed using human readable units (KB, MB, GB): ls -lh', '- Long format list sorted by size (descending): ls -ls', and '- Long format list of all files, sorted by modification date (oldest first): ls -ltr'. The prompt returns to '~ ' at the bottom.

```
~ tldr ls

ls

List directory contents.

- List files one per line:
  ls -l

- List all files, including hidden files:
  ls -a

- Long format list (permissions, ownership, size and modification date) of all
files:
  ls -la

- Long format list with size displayed using human readable units (KB, MB, GB)
:
  ls -lh

- Long format list sorted by size (descending):
  ls -ls

- Long format list of all files, sorted by modification date (oldest first):
  ls -ltr

~
```

This is not a substitute for `man` , but a handy tool to avoid losing yourself in the huge amount of information present in a man page. Then you can use the man page to explore all the different options and parameters you can use on a command.

ls

Inside a folder you can list all the files that the folder contains using the `ls` command:

```
ls
```

If you add a folder name or path, it will print that folder contents:

```
ls /bin
```



`ls` accepts a lot of options. One of my favorite options combinations is `-al`. Try it:

```
ls -al /bin
```

```
→ ~ ls -al /bin
total 5120
drwxr-xr-x@ 37 root wheel    1184 Feb  4 10:05 .
drwxr-xr-x  30 root wheel     960 Feb  8 15:32 ..
-rwxr-xr-x   1 root wheel   22704 Jan 16 02:21 [
-r-xr-xr-x   1 root wheel  618416 Jan 16 02:21 bash
-rwxr-xr-x   1 root wheel   23648 Jan 16 02:21 cat
-rwxr-xr-x   1 root wheel   34144 Jan 16 02:21 chmod
-rwxr-xr-x   1 root wheel   29024 Jan 16 02:21 cp
-rwxr-xr-x   1 root wheel  379952 Jan 16 02:21 csh
-rwxr-xr-x   1 root wheel   28608 Jan 16 02:21 date
-rwxr-xr-x   1 root wheel   32000 Jan 16 02:21 dd
-rwxr-xr-x   1 root wheel   23392 Jan 16 02:21 df
-rwxr-xr-x   1 root wheel   18128 Jan 16 02:21 echo
-rwxr-xr-x   1 root wheel   54080 Jan 16 02:21 ed
-rwxr-xr-x   1 root wheel   23104 Jan 16 02:21 expr
-rwxr-xr-x   1 root wheel   18288 Jan 16 02:21 hostname
-rwxr-xr-x   1 root wheel   18688 Jan 16 02:21 kill
-r-xr-xr-x   1 root wheel  1282864 Jan 16 02:21 ksh
-rwxr-xr-x   1 root wheel   121296 Jan 16 02:21 launchctl
```

compared to the plain `ls`, this returns much more information.

You have, from left to right:

- the file permissions (and if your system supports ACLs, you get an ACL flag as well)
- the number of links to that file
- the owner of the file
- the group of the file
- the file size in bytes
- the file modified datetime
- the file name

This set of data is generated by the `-l` option. The `-la` option instead also shows the hidden files.

Hidden files are files that start with a dot (`.`).

cd

Once you have a folder, you can move into it using the `cd` command. `cd` means **change directory**. You invoke it specifying a folder to move into. You can specify a folder name, or an entire path.

Example:

```
mkdir fruits  
cd fruits
```

Now you are into the `fruits` folder.

You can use the `..` special path to indicate the parent folder:

```
cd .. #back to the home folder
```

The `#` character indicates the start of the comment, which lasts for the entire line after it's found.

You can use it to form a path:

```
mkdir fruits  
mkdir cars  
cd fruits  
cd ../cars
```

There is another special path indicator which is `.`, and indicates the **current** folder.

You can also use absolute paths, which start from the root folder `/`:

```
cd /etc
```

This command works on Linux, macOS, WSL, and anywhere you have a UNIX environment

pwd

Whenever you feel lost in the filesystem, call the `pwd` command to know where you are:

```
pwd
```

It will print the current folder path.

mkdir

You create folders using the `mkdir` command:

```
mkdir fruits
```

You can create multiple folders with one command:

```
mkdir dogs cars
```

You can also create multiple nested folders by adding the `-p` option:

```
mkdir -p fruits/apples
```

Options in UNIX commands commonly take this form. You add them right after the command name, and they change how the command behaves. You can often combine multiple options, too.

You can find which options a command supports by typing `man <commandname>`. Try now with `man mkdir` for example (press the `q` key to esc the man page). Man pages are the amazing built-in help for UNIX.

rmmdir

Just as you can create a folder using `mkdir`, you can delete a folder using `rmdir`:

```
mkdir fruits  
rmdir fruits
```

You can also delete multiple folders at once:

```
mkdir fruits cars  
rmdir fruits cars
```

The folder you delete must be empty.

To delete folders with files in them, we'll use the more generic `rm` command which deletes files and folders, using the `-rf` options:

```
rm -rf fruits cars
```

Be careful as this command does not ask for confirmation and it will immediately remove anything you ask it to remove.

There is no **bin** when removing files from the command line, and recovering lost files can be hard.

mv

Once you have a file, you can move it around using the `mv` command. You specify the file current path, and its new path:

```
touch test
mv pear new_pear
```

The `pear` file is now moved to `new_pear`. This is how you **rename** files and folders.

If the last parameter is a folder, the file located at the first parameter path is going to be moved into that folder. In this case, you can specify a list of files and they will all be moved in the folder path identified by the last parameter:

```
touch pear
touch apple
mkdir fruits
mv pear apple fruits #pear and apple moved to the f
```

cp

You can copy a file using the `cp` command:

```
touch test  
cp apple another_apple
```

To copy folders you need to add the `-r` option to recursively copy the whole folder contents:

```
mkdir fruits  
cp -r fruits cars
```

open

The `open` command lets you open a file using this syntax:

```
open <filename>
```

You can also open a directory, which on macOS opens the Finder app with the current directory open:

```
open <directory name>
```

I use it all the time to open the current directory:

```
open .
```

The special `.` symbol points to the current directory, as `..` points to the parent directory

The same command can also be used to run an application:

```
open <application name>
```

touch

You can create an empty file using the `touch` command:

```
touch apple
```

If the file already exists, it opens the file in write mode, and the timestamp of the file is updated.

find

The `find` command can be used to find files or folders matching a particular search pattern. It searches recursively.

Let's learn it by example.

Find all the files under the current tree that have the `.js` extension and print the relative path of each file matching:

```
find . -name '*.js'
```

It's important to use quotes around special characters like `*` to avoid the shell interpreting them.

Find directories under the current tree matching the name "src":

```
find . -type d -name src
```

Use `-type f` to search only files, or `-type l` to only search symbolic links.

`-name` is case sensitive. use `-iname` to perform a case-insensitive search.

You can search under multiple root trees:

```
find folder1 folder2 -name filename.txt
```

Find directories under the current tree matching the name "node_modules" or 'public':

```
find . -type d -name node_modules -or -name public
```

You can also exclude a path, using `-not -path` :

```
find . -type d -name '*.md' -not -path 'node_modules'
```

You can search files that have more than 100 characters (bytes) in them:

```
find . -type f -size +100c
```

Search files bigger than 100KB but smaller than 1MB:

```
find . -type f -size +100k -size -1M
```

Search files edited more than 3 days ago

```
find . -type f -mtime +3
```

Search files edited in the last 24 hours

```
find . -type f -mtime -1
```

You can delete all the files matching a search by adding the `-delete` option. This deletes all the files edited in the last 24 hours:

```
find . -type f -mtime -1 -delete
```

You can execute a command on each result of the search. In this example we run `cat` to print the file content:

```
find . -type f -exec cat {} \;
```

notice the terminating `\; . {}` is filled with the file name at execution time.